

SCALA: Toward Imperceptible and Efficient Black-Box Textual Adversarial Perturbations

Siqi Sun^{ID}, Achim D. Brucker^{ID}, Jia Hu^{ID}, Xiaowei Huang^{ID}, *Senior Member, IEEE*, and Wenjie Ruan^{ID}

Abstract—Deep learning models are intrinsically susceptible to textual adversarial attacks on social media, where the perturbed text can trigger aberrant behaviours of victim models and threaten security and privacy. In this paper, we present a novel word-level attack called SCALA: a **S**ynonym-based **d**es**C**ending **A**nd **r**ep**L**ace-back **A**scending mechanism. Our focus is on the efficient production of adversarial examples, with a particular emphasis on minimizing human perceptibility while ensuring the visual resemblance and semantic correctness. The merits of our attacking solution lie in being: *i*) imperceptible – it keeps a very low word perturbation rate based on the Hamming (L_0 -norm) distance, thus achieving heightened deceptiveness validated through human evaluations; *ii*) efficient – our tensor-based parallelization strategy ensures the attacking efficiency compared with baselines; *iii*) effective – it surpasses seven state-of-the-art attacks on five target models in terms of reducing after-attack accuracy; *iv*) practical – black-box score-based setting ensures that the adversary only needs to query target models for confidence scores; and *v*) transferable – our attack shows competitive transferability on the generated adversarial examples. We release our code SCALA via <https://github.com/TrustAI/SCALA>

Index Terms—Model vulnerability, natural language processing, adversarial attacks, black-box setting, word-level perturbations.

I. INTRODUCTION

OVER the last few decades, deep neural networks (DNNs) have exhibited remarkable success in natural language processing (NLP) [1], [2], [3]. However, despite their impressive performance, DNNs have been shown to be highly vulnerable to adversarial examples, which are carefully crafted inputs with small, often imperceptible perturbations that cause models to make incorrect predictions. This vulnerability poses growing threats to security and privacy in online and safety-critical environments [4], [5], [6], [7].

The issue becomes even more serious with the widespread use of large language models (LLMs) such as GPT and LLaMA, which exhibit strong generalization and zero-shot learning abilities. Although these models revolutionize NLP tasks, they are not immune to adversarial manipulations. In particular, prompt-injection and jailbreak attacks can subvert

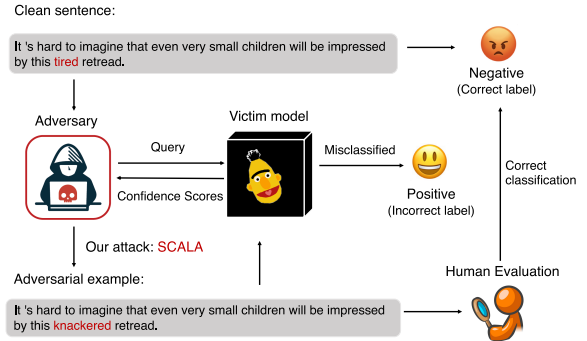


Fig. 1. Illustration of our effective black-box attack: in sentiment analysis for movie reviews, SCALA generates an adversarial example by querying the victim model to obtain confidence scores and subsequently substituting a word in the clean sentence with a synonym. This crafted adversarial text can deceive NLP models and evade detection from human evaluators.

safety alignment mechanisms, revealing a similar underlying weakness: subtle textual perturbations can drastically alter model behaviour [8], [9], [10]. Understanding and mitigating such vulnerabilities has therefore become a central research topic in Safe AI.

Compared to adversarial attacks in computer vision (CV) [6], [11], generating imperceptible adversarial examples for text presents key challenges. Images reside in continuous pixel spaces, where tiny numeric changes can easily be masked from human perception. Text, however, is inherently *discrete*, and even a single character or word change can significantly affect readability, grammar, or meaning [12]. Thus, designing textual perturbations that remain semantically faithful and grammatically correct, while still deceiving the model, is substantially more difficult [6], [13]. Character-level or sentence-level manipulations often yield unnatural text that can be detected by spell-checkers or simple rule-based defences [14], [15]. Conversely, word-level synonym substitution can produce more fluent and semantically consistent adversarial samples [16].

Most prior word-level adversarial methods concentrate on semantic preservation and grammaticality, but they paid little attention to how the perturbations are perceived visually by humans [16], [17], [18], [19]. Drawing inspiration from computer vision, we cast visual imperceptibility as a constraint on perturbation magnitude and operate within an L_p -norm framework [20]. Rather than using L_2 or L_∞ norms that are natural for continuous pixel spaces, we explicitly minimise the L_0 (Hamming) distance in token space because fewer token substitutions correlate more directly with human-perceived similarity. In this paper, we introduce **SCALA** (**S**ynonym-based **d**es**C**ending **A**nd **r**ep**L**ace-back **A**scending).

Received 14 August 2024; revised 10 April 2025 and 16 August 2025; accepted 20 October 2025. Date of publication 6 November 2025; date of current version 5 December 2025. The associate editor coordinating the review of this article and approving it for publication was Dr. Zhipeng Cai. (Corresponding author: Wenjie Ruan.)

Siqi Sun, Xiaowei Huang, and Wenjie Ruan are with the School of Computer Science and Informatics, University of Liverpool, L69 7ZX Liverpool, U.K. (e-mail: siqi.sun@liverpool.ac.uk; xiaowei.huang@liverpool.ac.uk; w.ruan@trustai.uk).

Achim D. Brucker and Jia Hu are with the Department of Computer Science, University of Exeter, EX4 4QJ Exeter, U.K. (e-mail: a.brucker@exeter.ac.uk; J.Hu@exeter.ac.uk).

Digital Object Identifier 10.1109/TIFS.2025.3629604

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: Bodleian Libraries of the University of Oxford. Downloaded on August 01, 2026 at 21:36:29 UTC from IEEE Xplore. Restrictions apply.

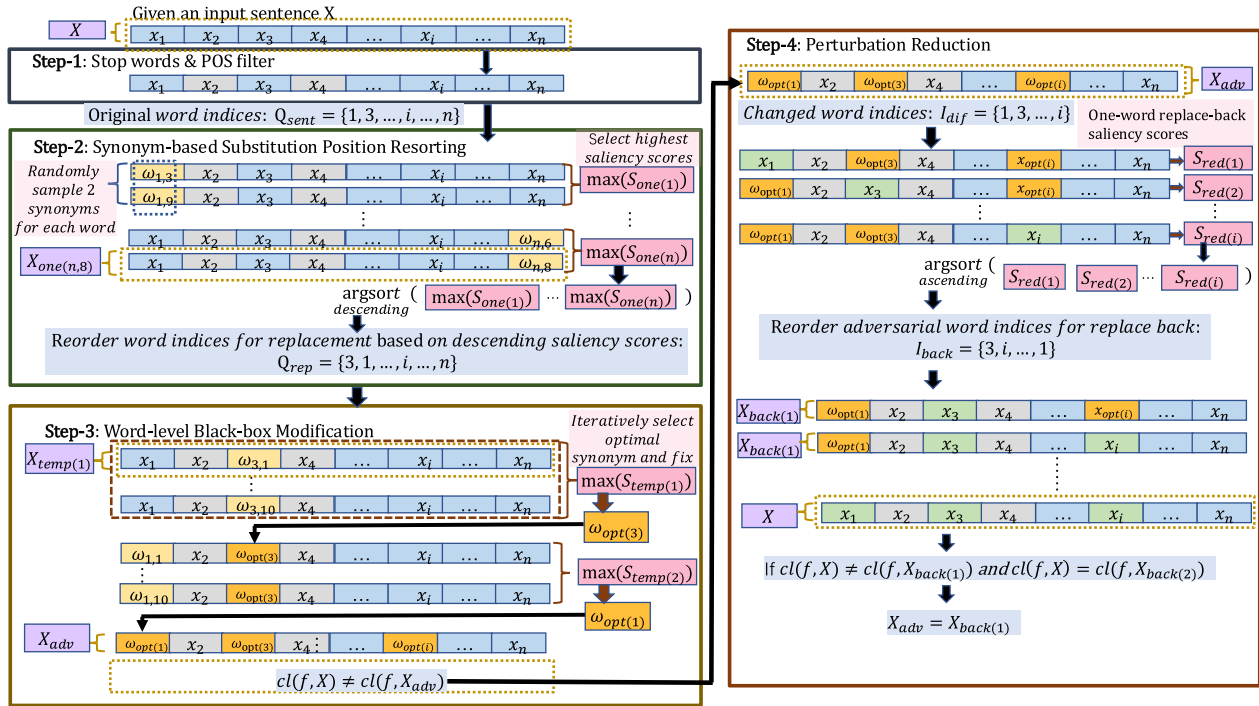


Fig. 2. Algorithmic overview of SCALA. Assume two synonyms are randomly selected for each word, in Step-2: purple: sentences in different states (X : original, X_{one} : one-word substitution, X_{temp} : accumulated perturbation, X_{adv} : adversarial example and X_{back} : adversarial example after replacing back), blue: original words, grey: filtered words, yellow: candidate synonyms, orange: local optimal synonyms, pink: saliency scores, green: replace-back words.

As Fig. 1 illustrates, SCALA first generates candidate synonym substitutions and ranks them using saliency derived from model confidence scores. After constructing a misclassifying sequence, a replace-back stage examines each substituted token and restores those that are redundant for the attack, thereby reducing the final perturbation rate. To mitigate the high query and runtime cost of iterative score-based attacks, especially on long documents, SCALA includes a tensorized parallelization module that evaluates many one-word variants in batch. This design significantly decreases generation time while preserving attack effectiveness.

Finally, while adversarial work on LLMs has focused predominantly on jailbreak and prompt-injection attacks [8], [10], [21], we shift attention to structured classification: we evaluate how fine-tuning LLMs for classification tasks affects their susceptibility to word-level synonym substitutions, and demonstrate that SCALA can produce highly-concealed yet transferable adversarial examples. The key contributions of this paper are summarised below.

(i) Our attack based on Hamming distance and a flexible and practical *double-sorting* synonym-level mechanism can effectively ensure a *limited-quantity* modifications and *visually similar* adversarial examples. SCALA can work in a black-box setting where only the model's confidence scores are accessible.

(ii) We evaluate SCALA on *five* well-established pre-trained NLP victim models over multiple NLP tasks including sentiment analysis, question answering, multi-categorization of news articles, and entailment relationship inference. Compared to prior works, it achieves a *higher* attacking success rate, *lower* perturbation rate, *imperceptible* grammatical errors under both software checker and human judges.

(iii) We also optimize our black-box attack through designing a *parallelization module*, which leads to a *superior efficiency*, e.g., it can generate an adversarial sentence within *2.5 seconds* for IMDB dataset, *2 to 1,100 times* faster than state-of-the-art baselines. Our attack further shows *better* transferability and *improved* robustness in adversarial training, which is beneficial for conducting further studies.

(iv) We conduct investigation in evaluating fine-tuned LLMs under adversarial attacks in classification settings, extending beyond the existing focus on jailbreaking and prompt injection. Our findings provide new insights into the robustness of LLMs when adapted for structured NLP tasks.

II. RELATED WORK

We review existing textual adversarial attacks from the following three aspects.

A. White-Box and Transfer-Based Attacks

Early NLP adversarial research adopted **white-box** settings inspired by computer vision, assuming full access to model parameters. Representative methods such as HotFlip and TextBugger compute gradients or Jacobians with respect to text edits to craft perturbations [14], [22], [23], [24]. Subsequent works explored geometry-based or loss-guided strategies [9], [16], [19], [25]. However, these attacks are unrealistic when the model architecture or weights are unavailable. **Transfer-based attacks** [26] train a substitute model to approximate the target, but high training cost and limited transferability restrict their practicality.

B. Black-Box Attacks

Black-box attacks assume that only the model's outputs are observable. **Hard-label attacks** access only the predicted label

and rely on heuristic or evolutionary algorithms to explore the discrete input space. Genetic algorithms, population-based search, and query-efficient optimization have been widely used [9], [18], [27]. These methods achieve reasonable success but require large numbers of model queries and are computationally expensive. **Soft-label attacks** exploit confidence scores to guide fine-grained perturbation ranking. Representative examples include GA [18], PWWS [16], PSO [27], TextFooler [17], and BERT-Attack [19]. Later variants improved semantic consistency through synonym selection or language-model masking but remain iterative and slow on long sequences [28]. However, very few studies explicitly address the human perceptibility or visual similarity of adversarial text, leading to perturbations that are easily noticeable [20].

C. Adversarial Attacks on LLMs

Recent work has extended adversarial research to LLMs, mainly through *jailbreak* and *prompt-injection* attacks [8], [9], [10]. Unlike traditional text-classification attacks, these attacks attempt to bypass safety constraints or manipulate model behavior. **Prompt injection** circumvents safety mechanisms by embedding adversarial instructions within user prompts, potentially leading to data leakage or harmful outputs. Representative methods include goal hijacking and prompt leaking [29], black-box prompt injection [30], and template-based exploits of instruction-tuned structures [31]. **Jailbreak attacks** override predefined restrictions to elicit disallowed responses. Early examples such as Do Anything Now (DAN) [32] condition models to ignore safeguards, while automated systems including MasterKey [33], GPTFuzz [34], and AutoDAN [35] systematically generate prompts that bypass alignment. Most existing studies emphasize safety circumvention in conversational settings rather than robustness in structured classification. Our work instead examines *word-level adversarial perturbations for fine-tuned LLMs*, an area still largely unexplored.

In conclusion, our work belongs to the family of soft-label (score-based) adversarial attacks for text classification. Unlike existing approaches, we balance practicality, imperceptibility, and efficiency through a novel score-based descending and replace-back ascending mechanism that ranks synonyms and removes redundant substitutions, yielding less perceptible perturbations. A parallelized computation module further accelerates generation. Beyond conventional classification settings, we take the first step in evaluating fine-tuned LLMs under word-level adversarial attacks, extending adversarial research to LLM-based structured classification tasks.

III. METHODOLOGY

A. Problem Formulation

Given an input sentence X with n words from a corpus dataset. Let $X = \{x_i \mid i \in \mathbb{N}_{1 \leq n}^*\}$, where x_i denotes the i th word in the original sentence. A target model f is pre-trained to learn a mapping from the input X to a prediction label Y . Specifically, the victim model f computes confidence scores $C(X)$ for the input. Thus, the final classification label is represented by $Y = cl(f, X) = \text{*argmax}_{l \in \{1, \dots, m\}} C_l(X)$ with the highest confidence score $C_l(X)$ selected from m labels. In our score-based black-box setting, the adversary is unaware of the model architecture, parameter settings, or training data, and

can only query the target model with supplied inputs to get the output prediction scores.

Norm Selection: L_0 Norm. In our strategy, the L_0 norm (Hamming distance) is adopted. Given an original sentence X and a corresponding adversarial sentence X_{adv} , the L_0 distance is denoted as $\|X_{adv} - X\|_0$, which measures the number of words that differ between the two sentences. The L_0 norm is particularly well-suited for textual adversarial attacks because it directly quantifies the number of discrete token substitutions, which is both interpretable and relevant to human perception. In robustness certification literature, L_0 has been theoretically analyzed for synonymous word substitution [36], [37], [38], providing provable bounds in discrete token space. In contrast, the L_2 norm, commonly used in continuous embedding spaces, where small L_2 shifts can produce large textual changes after decoding. In SCALA, minimizing L_0 is a primary design goal, as smaller L_0 distances typically yield adversarial examples that are perceptibly closer to the original sentence when preserving the semantic similarity.

Constraint-optimization Problem. For a given input X , we aim to craft X_{adv} by word-level synonym substitution. Specifically, the generation of a valid adversarial sequence X_{adv} can be formulated as a constrained optimization [39]:

$$\min_{X_{adv}} \|X_{adv} - X\|_0 \text{ s.t. } cl(f, X) \neq cl(f, X_{adv}) \\ \text{and } d(X_{adv}, X) \geq \epsilon \quad (1)$$

The objective is to generate an adversarial sentence X_{adv} that minimizes its Hamming distance to original sentence X , i.e., $\|X_{adv} - X\|_0$. This promotes minimal and discrete perturbations, ensuring syntactic imperceptibility. In parallel, X_{adv} is required to alter the prediction of the target classifier (i.e., $cl(f, X) \geq cl(f, X_{adv})$), ensuring the misclassification.

Semantic constraint. To ensure semantic fidelity, we use the Universal Sentence Encoder (USE) [40] to measure similarity between X and X_{adv} , following prior work [17], [19]. Specifically, we define $d(\cdot)$ as the cosine similarity between the USE embeddings of the two texts. In contrast to prior research, our approach prioritizes the reduction of perturbation rates, aiming to make the generated sentences less perceptible to the human eye with minimal changes to their original positions. Additionally, we also take into account semantic similarity to preserve the naturalness of the sentences. However, note that the problem defined here is NP-hard owing to the complexity and non-convex of deep NLP models [9]. If we directly adopt exhaustive search, the computation complexity is growing exponentially thus not practically usable. Thus, we propose an approximation-based solution to maintain both the small perturbation and high efficiency.

B. SCALA: SYnonym-Based DesCEnding AND RepLAcE-Back AScending Mechanism

The overview of SCALA is shown in **Fig. 2**, **Alg. 1** and **Alg. 2**, where four steps constitute the generation and perturbation reduction of an adversarial sentence. **Step-1** filters out part-of-speech mismatched and stop words in an original input X and obtains a set Q_{sent} with sentence-order word indices. **Step-2** samples synonyms to create X_{one} with one-word substituted and resorts a set Q_{rep} with word indices based on saliency scores S_{one} in descending order. **Step-3** iteratively generates X_{temp} with accumulated perturbations based

on Q_{rep} until an adversarial sentence X_{adv} is created. Finally, **Step-4** computes saliency scores via one-word replace-back sentences X_{back} and reorders word indices in ascending order. A redundancy-eliminated adversarial sentence X_{adv} is obtained by accumulatively replacing back words.¹

Step-1: Filter Rules & Synonym Selection (Line 1-3, Alg. 1)

We first identify candidate words for substitution by filtering out common low-content or non-content words, those that do not significantly affect the overall meaning of the sentence. These include articles (e.g., *a, an, the*), prepositions (e.g., *in, on, at*), conjunctions, numbers, and frequent stop words. We adopt predefined stop word lists from the NLTK² and spaCy³ libraries. Specifically, we tokenize each sentence and normalize the tokens by converting them to lowercase. Any word that matches an entry in either the NLTK or spaCy stop word lists is excluded from the substitution candidates. This ensures that substitution candidates are semantically meaningful and grammatically impactful, which improves the fluency and imperceptibility of the adversarial examples. Given an input sentence X with n tokens, we define the set of eligible word indices for substitution as $Q_{sent} \subseteq \{1, \dots, n\}$, where only non-stop words are included. This filtered set is then used in our saliency-based ranking and substitution process. Following this, we select a set of candidate synonyms for each substitution position in Q_{sent} via the counter-fitted embedding space [41]. Then, considering an original word x_i , $Z_{syn(i)}$ is represented as a set including all the synonym candidates for x_i :

$$Z_{syn(i)} = \{\omega_{i,j} \mid i \in Q_{sent} \wedge \forall j \in \mathbb{N}_{j \leq |Z_{syn(i)}|}^*\}, \quad (2)$$

where $\omega_{i,j}$ is a single synonym among the set $Z_{syn(i)}$, here i denotes the index in the set Q_{sent} with original word indices and j denotes the index in the synonym set $Z_{syn(i)}$.

Step-2: Synonym-based Substitution Position Resorting (Line 4-19 in Alg. 1)

In this step, we aim to reasonably sort candidate positions for substitution in the original sentence, a synonym-based reorder strategy for the set Q_{sent} with sentence-order indices is provided. To ensure the higher efficiency especially for datasets with over-length sentences like IMDB, we also introduce a random sampling mechanism to sample the candidate synonyms for each word x_i . Specifically, given a tunable parameter τ , a set of replacement indices Q_{sent} , the synonym set $|Z_{syn(i)}| = z$, a randomized sampling-based synonym set $Z_{ran(i)}$ for original word x_i is defined as below:

$$Z_{ran(i)} = \{\omega_{i,j} \mid i \in Q_{sent}, j \in \text{random}_{\tau * z} Z_{syn(i)}\}, \quad (3)$$

where a randomly selected synonym $\omega_{i,j} \in Z_{syn(i)}$. The function random is utilized to randomly select the fixed number of $|Z_{ran(i)}| = \tau * z$ synonyms. For example, assume that each original word has $z = 10$ candidate synonyms, let $\tau = 0.2$, then 2 candidates are randomly selected for each eligible substitution position. On this basis, a sentence X_{one} with one-synonym perturbation is constructed based on the original sentence X as below:

$$X_{one(i,j)} = \{x_1, x_2, \dots, \omega_{i,j}, \dots, x_n\}, \quad (4)$$

¹Note that the use of mathematical sets is intended to enhance the convenience of explaining the parallelization module in the next subsection.

²<https://www.nltk.org/>

³<https://github.com/explosion/spaCy>

Algorithm 1 The Generation of Word-level Adversarial Attack (Step 1-3)

Input: Input sentence $X = \{x_1, x_2, \dots, x_n\}$, target model f , prediction label Y , ground true label Y^* , confidence score C , compute classification label cl , a corpus $Z_{syn(i)}$ includes z candidate synonyms for each word.

Parameter: τ : limit the number of synonyms

Output: Adversarial example X_{adv}

```

1: for word  $x_i$  in  $X$  do
2:   Set of filtered word indices:
    $Q_{sent} \leftarrow \text{POSFilter}(\text{all word indices})$ 
3: end for
4: for sentence-order word index  $i$  in  $Q_{sent}$ : do
5:   for synonym  $\omega_{i,j}$  in  $\tau * z$  randomly sampled synonyms
   in  $Z_{syn(i)}$  do
6:     Initialize one-word substitution sentence  $X_{one} \leftarrow X$ 
7:     In  $X_{one}$ : word  $x_i \leftarrow \omega_{i,j}$ 
8:     if  $Y = Y^*$  then
9:       Saliency score  $S_{one(i,j)} \leftarrow C_{Y^*}(X) - C_{Y^*}(X_{one(i,j)})$ 
10:    else
11:      Saliency score  $S_{one(i,j)} \leftarrow C_{Y^*}(X) - C_{Y^*}(X_{one(i,j)}) +$ 
       $C_Y(X_{one(i,j)}) - C_Y(X)$ 
12:    end if
13:  end for
14:  Local optimal score:  $S_{opt(i)} \leftarrow \max(S_{one(i)})$ 
15: end for
16: Indices for saliency scores in a descending order:
    $Q_{rep} \leftarrow \text{argsort}_{\text{descending}} S_{opt}$ 
17: for  $i$  in  $Q_{rep}$  do
18:   Reordered word indices:  $Q_{rep} \leftarrow Q_{sent(i)}$ 
19: end for
20: Initialize accumulated-perturbed sentence  $X_{temp} \leftarrow X$ ,
   iteration number  $k = 1$  and  $k \leq |Q_{rep}|$ .
21: for substitution index  $i$  in  $Q_{rep}$ : do
22:   In  $X_{temp(k)}$ : word  $x_i \leftarrow \omega_{opt(i)}$ 
23:   if  $cl(f, X) \neq cl(f, X_{adv})$  then
24:     Target model  $f$  is fooled
25:     The adversarial example is generated:  $X_{adv} \leftarrow$ 
      $X_{temp(k)}$ 
26:   break
27:   else
28:     Iteration number  $k = k + 1$ 
29:   end if
30: end for
```

where only one original word x_i is replaced with a synonym $\omega_{i,j} \in Z_{ran(i)}$ (3), here the original word index $i \in Q_{sent}$ and synonym index $j \in \mathbb{N}_{j \leq |Z_{ran(i)}|}$. Following this, the saliency score $S_{one(i,j)}$ between an input X and a one-word substitution sentence $X_{one(i,j)}$ is computed as:

$$S_{one(i,j)} = \begin{cases} C_{Y^*}(X) - C_{Y^*}(X_{one(i,j)}), & \text{if } Y = Y^* \\ C_{Y^*}(X) - C_{Y^*}(X_{one(i,j)}) + C_Y(X_{one(i,j)}) - C_Y(X), & \text{otherwise} \end{cases} \quad (5)$$

where C returns the confidence score under the prediction label. If the label for $X_{one(i,j)}$ is different from the input X , i.e., $Y \neq Y^*$, the saliency score $S_{one(i,j)}$ is increased with more importance. Note that a higher saliency score here

Algorithm 2 Perturbation Reduction for Textual Redundancies (Step 4)

Input: Input sentence $X = \{x_1, x_2, \dots, x_n\}$, target model f , ground true label Y , confidence score C , compute classification label cl , a set I_{dif} includes different word indices between the adversarial sentence X_{adv} and the input X .

Output: Adversarial example X_{adv} with perturbation reduction

- 1: **for** word index i in I_{dif} **do**
- 2: Initialize one-word replace-back sentence $X_{red} \leftarrow X_{adv}$
- 3: In X_{red} : $x_{adv(i)} \leftarrow x_i$
- 4: **if** $Y = Y^*$ **then**
- 5: Saliency score $S_{red(i)} \leftarrow C_{Y^*}(X_{adv}) - C_{Y^*}(X_{red(i)})$
- 6: **else**
- 7: Saliency score $S_{red(i)} \leftarrow C_{Y^*}(X_{adv}) - C_{Y^*}(X_{red(i)}) + C_Y(X_{red(i)}) - C_Y(X_{adv})$
- 8: **end if**
- 9: **end for**
- 10: Reorder indices for saliency scores in an ascending order: $\text{argsort}_{\text{ascending}} S_{red}$
- 11: **for** i in $\text{argsort}_{\text{ascending}} S_{red}$ **do**
- 12: Reordered word indices for redundancy reduction: $I_{back(i)} \leftarrow I_{dif(i)}$
- 13: **end for**
- 14: **for** word index i in I_{back} **do**
- 15: In X_{adv} : $x_{adv(i)} \leftarrow x_i$
- 16: **if** $cl(f, X_{adv}) \neq cl(f, X)$ **then**
- 17: Continue
- 18: **else**
- 19: Remain $x_{adv(i)}$ unchanged in X_{adv}
- 20: **end if**
- 21: **end for**

indicates a more competitive candidate synonym that results in misclassification for victim models. On the contrary, a lower or even minus saliency score represents limited or even adverse influence on fooling deep classifiers. Therefore, if i th word x_i being replaced by j th synonym $\omega_{i,j}$ in $Z_{ran(i)}$ causes the most significant change in the classification probability, an optimal synonym $\omega_{opt(i)} = \omega_{i,j}$ is selected for substitution. This created sentence is denoted as $X_{opt(i)}$. Therefore, the local optimal saliency score $S_{opt(i)}$ is defined as:

$$\exists X_{opt(i)} \text{ s.t. } S_{opt(i)} = C_Y(X) - C_Y(X_{opt(i)}) \quad (6)$$

Furthermore, a merging set S_{opt} made up of individual optimal saliency score $S_{opt(i)}$ is obtained: $S_{opt} = \{S_{opt(i)} \mid \forall i \in Q_{sent}\}$. Thus, the descending order for saliency scores helps construct a reasonable sort Q_{rep} for word replacement by reordering indices in sentence-order set Q_{sent} as follows:

$$Q_{rep} = \{Q_{sent(i)} \mid i \in \text{argsort}_{\text{descending}} S_{opt}\}. \quad (7)$$

Step-3: Word-level Adversarial Example Generation (Line 20-30 in Alg. 1)

In this step, a word-level modification mechanism is designed based on the reasonable sort Q_{rep} in (7). $X_{temp(k)}$ is created as a sentence that iteratively constructs the i -th word

substitution in k th iteration such that $k \leq |Q_{rep}|$. The substitution is realized with a local optimal synonym $\omega_{opt(i)} \in Z_{sym(i)}$ defined in (2), which results in the highest saliency score in each iteration. Noted that this word substitution is fixed, and the next substitution is continued based on this fixed sequence until the target model makes an incorrect prediction. Thus, the sentence $X_{temp(k)}$ with accumulated perturbations in k th iteration is denoted as below:

$$X_{temp(k)} = \{(X_{temp(k-1)} \setminus x_i) \cup \omega_{opt(i)} \mid i = Q_{rep(k)}\}, \quad (8)$$

where the sentence $X_{temp(k-1)}$ is generated in the last iteration. We replace the original word x_i with the local optimal synonym $\omega_{opt(i)}$, noted that here i means the k th index in the set Q_{rep} with original word indices.

Then the target model is queried with $X_{temp(k)}$ for the confidence score and label. The model is queried continuously until a model misclassification appears. Here the obtained adversarial sentence is denoted as X_{adv} .

To ensure that the generated adversarial example X_{adv} remains semantically similar to the original input, we compute the semantic similarity between X and $X_{temp(k)}$ using the USE [40]. We retain only those intermediate or final adversarial sequences where the USE-based cosine similarity score exceeds a predefined threshold ϵ . That is, we accept $X_{temp(k)}$ or X_{adv} only if $\text{sim}(X, X_{temp(k)}) \geq \epsilon$. This filtering ensures that all generated adversarial examples preserve semantic fidelity while successfully misleading the NLP model. The final adversarial sentence, denoted as X_{adv} , satisfies the misclassification condition:

$$\exists X_{adv} \text{ s.t. } cl(f, X) \neq cl(f, X_{adv}) \text{ and } d(X, X_{adv}) \geq \epsilon. \quad (9)$$

Step-4: Perturbation Reduction for Textual Redundancies (Alg. 2)

After crafting the adversarial example, it is worth noting that not all the modifications in X_{adv} are required to make an incorrect prediction. Thus, the elimination of the redundancies leads to perturbation reduction, which aims to shrink the Hamming distance to ensure that the adversarial text is more imperceptible to human beings.

Assume that the set for different word indices between the adversarial sentence X_{adv} and the input X is denoted as I_{dif} . Similar to vulnerable word sorting in step-2, a different word in adversarial sentence X_{adv} is replaced back to an original word. This one-word replace-back sentence with redundancies is denoted as $X_{red(i)}$:

$$X_{red(i)} = \{(X_{adv} \setminus x_{adv(i)}) \cup x_i \mid i \in I_{dif}\}, \quad (10)$$

where the word $x_{adv(i)}$ in X_{adv} is replaced back to the original word x_i and i is an index in the set I_{dif} . Similar with (5), the saliency score between X_{adv} and $X_{red(i)}$ is computed:

$$S_{red(i)} = \begin{cases} C_{Y^*}(X_{adv}) - C_{Y^*}(X_{red(i)}), & \text{if } Y = Y^* \\ C_{Y^*}(X_{adv}) - C_{Y^*}(X_{red(i)}) + C_Y(X_{red(i)}) - C_Y(X_{adv}), & \text{otherwise} \end{cases} \quad (11)$$

where $i \in I_{dif}$. Here a lower saliency score indicates a weaker influence on misleading victim models and corresponds to a more appropriate word to be substituted back. The saliency scores $S_{red(i)}$ constitute a set $S_{red} \in \mathbb{R}^{|I_{dif}|}$.

Thus the indices of replace-back words are reordered and a reasonable sort I_{back} in ascending order for perturbation redundancy is obtained:

$$I_{back} = \{I_{dif(i)} \mid i \in \text{argsort}_{\text{ascending}} S_{red}\}. \quad (12)$$

Based on the ascending sort I_{back} , the perturbed words in X_{adv} are iteratively substituted back to the original word. The replace-back words are fixed until the last adversarial word is returned. Assume that $X_{back(i)}$ is the sentence with i adversarial words replaced back to original ones based on the set of reordered indices $\{I_{back(1)}, \dots, I_{back(i)}\}$ in i th iteration. Following this, check whether $cl(f, X_{back(i)}) \neq cl(f, X)$. If any element replacement associated with $X_{back(i)}$ leads to true, we conclude that the corresponding words are unnecessary for misclassification. Thus, the last sentence $X_{back(k)}$ is selected as the final adversarial example if it can result in misprediction with as many replace-back words as possible in the ascending order I_{back} . The indices for substituting back are $\{I_{back(1)}, \dots, I_{back(k)}\}$. Conversely, if all the sentences X_{back} are false for $cl(f, X_{back(k)}) \neq cl(f, X)$, we reserve all the modifications for X_{adv} . Consequently, an adversarial sentence X_{adv} with a small quantity of alterations is created.

C. Parallelization Module Design

We further improve our algorithm SCALA to be more applicable to efficiency-demanding scenarios where the number of synonyms is high, or massive amounts of data in the dataset. We utilize tensor-based parallelization to compute the saliency scores simultaneously. Unlike previous work [17], [19], [42] that relies on iterative computation for each substitution, our approach enables simultaneous evaluation of all variants in a batched and parallelized manner, significantly reducing computational overhead. Specifically, the one-word synonym-replacement sentence X_{one} (4), the potential adversarial example X_{temp} (8), one-word replace-back sentence X_{red} (10) and iterative replace-back sentence X_{back} are combined as a whole list respectively to feed into the NLP classifier and to obtain the confidence scores. Following this, importance scores $S_{one(i,j)}$ (5) for one-word substitution and $S_{red(i)}$ (11) for one-word replace-back are further computed as multi-dimensional arrays parallelly. This parallelization strategy is a deliberate algorithmic improvement that allows SCALA to maintain high efficiency even in long-text or large-scale settings, where prior approaches typically degrade. It is a key reason why SCALA can be readily applied to real-world, efficiency-sensitive tasks.

IV. EXPERIMENTS

A. Experimental Settings

1) **Datasets:** We evaluate SCALA on seven benchmark datasets that cover representative NLP tasks, including sentiment analysis, news categorization, question answering and textual entailment. Table I summarizes the dataset statistics.

Text classification. We use five standard benchmarks: MR [43], IMDB [44], and Yelp [45] for binary sentiment analysis of movie and user reviews; AG News [45] for four-way news categorization based on article titles and descriptions; and BoolQ [46] as a yes/no question-answering task framed as binary classification for further LLM attack evaluation.

Textual entailment. We further adopt SNLI [47] and MultiNLI [48], which contain sentence pairs annotated as entailment, contradiction, or neutral, enabling evaluation of cross-genre inference.

2) **Victim Models:** We focus on the following target models.

TABLE I

STATISTICS OF DATASETS. TRAIN: NUMBER OF TRAINING DATASET, TEST: NUMBER OF TEST DATASET, AVG. LEN.: THE AVERAGE LENGTH OF SENTENCES, LAB.: NUMBER OF CLASSIFICATION LABELS

Task	Dataset	Train	Test	Avg. Len.	Lab.
Text Classification	MR	9K	1K	20	2
	IMDB	25K	25K	215	2
	Yelp	560K	38K	152	2
	AG	120K	7.6K	43	4
	BoolQ	9.4K	3.2K	128	2
Textual	SNLI	570K	3K	8	3
Entailment	MultiNLI	433K	10K	11	3

(i) **LLaMA-3.2-1B and LLaMA-3.2-3B:** Both models are fine-tuned using the HuggingFace Transformers framework [49] for binary and multi-class classification. Inputs are tokenized and truncated to 512 tokens, optimized with AdamW (learning rate 1e-5, batch size 16) for 10 epochs under a linear warm-up schedule (40% warm-up steps). (ii) **BERT Base-uncased:** A 12-layer transformer with 768 hidden units and 12 attention heads (110M parameters) [50]. (iii) **WordCNN:** A convolutional model [51] with 150 filters and 3–5-gram windows. (iv) **WordLSTM:** A single-layer bidirectional LSTM containing 150 hidden units.

For textual entailment, BERT base-uncased model is also adopted. Additionally, we also include: (i) **ESIM:** An enhanced model for natural language inference based on chain LSTM [52]. (ii) **InferSent:** An English sentence encoder for universal sentence embeddings, trained on the supervised data of SNLI dataset [53].

3) **Baselines:** We have selected seven representative baselines. (i) **Genetic Attack:** A black-box attack adopting a population-based optimization algorithm [18]. (ii) **TextBugger:** In the black-box scenario, it finds important sentences and then uses a scoring function to determine which words are important to manipulate. [14]. (iii) **TextFooler:** This work ranks words based on important scores and considers sentence similarity scores to craft attacks [17], we compare this attack under both tasks of text classification and entailment. (iv) **PSO:** Sememe-based word substitution method by using particle swarm optimization [27]. (v) **PWWS:** This attack determines the order of named entity substitution based on word saliency and weighted by classification probability [16]. (vi) **Bert-Attack:** It utilizes the masked language model to find perturbations that increase the risk of making wrong predictions for target models [19]. BAE is not on the list since it performs similarly to BERT-Attack [24]. (vii) **Hard-label Attack:** It adopted a genetic algorithm to generate adversarial examples in a hard-label setting [42]. The experiments are performed utilizing TextAttack [54]. (viii) **ABP:** Adversarial Boosting Preference is a recent black-box ranking-and-prune pipeline [28], which we adopt for the LLM attack in our experiments.

4) **Parameter Settings:** In our experiments, a NLTK filter is used to filter stop words, while Spacy tags are used for POS tags. In accordance with the procedures discussed by [18], we test our algorithm with randomly selected 1,000 examples from the test set in each dataset. These examples are taken following operations as [17] for a fair comparison. The number z of synonyms for each word is set to **50** and the parameter

TABLE II

TASK OF TEXT CLASSIFICATION IN CONTRAST WITH 7 BASELINES (2ND COLUMN) OVER 4 DATASETS (1ST COLUMN) FOR 3 CLASSIFIERS (1ST ROW). THE ABBREVIATIONS: ORIGINAL ACCURACY(Orig.%), AFTER ATTACK ACCURACY(Acc.%) AND PERTURBATION RATE (Pert.%), GRAM.% (INCREASING GRAMMATICALITY ERROR RATE), ↓ REPRESENTS THAT THE LOWER THE BETTER

Dataset	Attack	Bert				WordLSTM				WordCNN			
		Orig.%	Acc.% ↓	Pert.% ↓	Gram.% ↓	Orig.%	Acc.% ↓	Pert.% ↓	Gram.% ↓	Orig.%	Acc.% ↓	Pert.% ↓	Gram.% ↓
MR	GA		32.00%	17.39%	12.68%		17.00%	18.39%	13.81%		11.00%	18.49%	11.68%
	TextBugger		35.30%	13.89%	35.72%		18.40%	13.20%	27.53%		18.80%	14.41%	34.92%
	Hard-label Attack		2.00%	13.34%	6.36%		2.70%	13.58%	4.07%		2.30%	12.02%	8.07%
	PSO	96.90%	6.00%	20.21%	3.58%	86.60%	4.30%	14.27%	2.23%	85.40%	4.20%	15.47%	4.80%
	BERT-Attack		23.22%	16.09%	13.51%		8.80%	16.53%	13.62%		10.10%	17.29%	16.55%
	PWWS		13.70%	15.20%	6.29%		5.60%	12.86%	6.99%		4.30%	13.24%	8.73%
	TextFooler		7.10%	18.57%	9.17%		1.00%	13.08%	5.79%		0.50%	14.06%	7.16%
	Ours		2.10%	10.39%	4.44%		0.20%	9.32%	3.49%		0.00%	9.70%	6.33%
IMDB	GA		46.00%	5.31%	1.93%		13.00%	5.23%	1.88%		6.00%	5.26%	1.73%
	TextBugger		11.00%	8.68%	11.78%		0.40%	4.33%	4.56%		0.20%	4.06%	4.88%
	Hard-label Attack		1.00%	3.65%	2.20%		1.00%	3.27%	1.74%		0.00%	3.32%	2.33%
	PSO	89.10%	1.30%	3.70%	2.00%	89.20%	0.00%	2.76%	0.39%	88.10%	0.00%	3.01%	0.61%
	BERT-Attack		9.50%	5.55%	1.99%		0.70%	4.73%	1.01%		0.10%	4.19%	1.17%
	PWWS		2.50%	4.68%	1.46%		0.20%	2.33%	0.75%		0.00%	2.04%	0.74%
	TextFooler		1.30%	8.54%	3.11%		0.00%	2.69%	0.93%		0.00%	2.46%	0.92%
	Ours		0.00%	2.28%	1.07%		0.00%	1.55%	0.96%		0.00%	1.56%	1.37%
Yelp	GA		24.00%	11.34%	5.62%		5.00%	7.72%	3.02%		18.00%	7.52%	2.98%
	TextBugger		14.60%	10.43%	13.54%		4.40%	8.02%	7.06%		5.80%	8.86%	8.98%
	Hard-label Attack		5.00%	6.37%	5.16%		3.00%	5.19%	2.77%		1.00%	6.12%	4.09%
	PSO	94.60%	1.00%	11.04%	0.44%	94.00%	0.00%	7.07%	0.70%	94.00%	1.00%	7.48%	0.35%
	BERT-Attack		7.91%	8.09%	3.14%		2.40%	8.62%	1.44%		3.60%	8.85%	1.66%
	PWWS		6.90%	6.92%	1.91%		0.90%	4.97%	1.01%		1.10%	5.73%	1.28%
	TextFooler		5.80%	9.76%	3.23%		0.30%	5.61%	1.44%		0.40%	6.38%	1.91%
	Ours		1.00%	4.43%	1.78%		0.00%	3.60%	1.68%		0.20%	4.34%	2.58%
AG	GA		46.00%	16.55%	7.09%		41.00%	15.46%	6.89%		39.00%	18.19%	6.94%
	TextBugger		42.00%	18.58%	26.84%		22.30%	19.00%	20.91%		11.80%	17.27%	20.84%
	Hard-label Attack		5.80%	12.20%	5.50%		5.70%	13.20%	4.08%		1.40%	10.54%	5.63%
	PSO	94.60%	20.60%	16.70%	1.46%	89.90%	10.90%	20.45%	2.45%	90.70%	8.10%	20.03%	2.26%
	BERT-Attack		35.13%	17.97%	0.61%		31.60%	18.31%	0.95%		17.70%	19.80%	2.07%
	PWWS		37.80%	17.20%	2.44%		14.60%	16.59%	3.62%		7.80%	14.54%	4.65%
	TextFooler		16.10%	22.82%	8.00%		4.70%	17.92%	6.74%		0.70%	15.37%	6.06%
	Ours		4.30%	10.17%	2.38%		0.90%	10.62%	6.44%		0.00%	7.86%	5.73%

TABLE III

TEXT ENTAILMENT TASK COMPARED WITH TEXTFOOLER. ABBREVIATIONS: ORIG.% (ORIGINAL ACCURACY), ACC.% (AFTER ATTACK ACCURACY), PERT.% (PERTURBATION RATE), GRAM.% (GRAMMATICALITY). ↓ DENOTES LOWER VALUES ARE BETTER

Dataset	Attack	Bert				InferSent				ESIM			
		Orig.%	Acc.% ↓	Pert.% ↓	Gram.% ↓	Orig.%	Acc.% ↓	Pert.% ↓	Gram.% ↓	Orig.%	Acc.% ↓	Pert.% ↓	Gram.% ↓
SNLI	TextFooler	89.1%	1.70%	7.18%	36.4%	79.0%	6.50%	5.86%	31.3%	86.5%	5.10%	9.05%	25.5%
	Ours		0.40%	5.36%	17.2%		1.70%	5.74%	11.0%		1.10%	5.42%	8.8%
MultiNLI(matched)	TextFooler	85.1%	5.30%	7.03%	29.4%	67.3%	8.00%	5.32%	17.3%	78.1%	7.70%	7.25%	20.1%
	Ours		0.50%	4.68%	10.8%		0.70%	5.11%	9.90%		0.50%	4.40%	9.30%
MultiNLI(mismatched)	TextFooler	85.1%	3.40%	6.68%	28.90%	62.4%	6.90%	5.74%	22.10%	75.5%	7.30%	7.30%	22.40%
	Ours		0.30%	4.43%	11.5%		0.40%	4.69%	8.70%		0.10%	4.20%	11.40%

τ in (3) is set to **0.06** and **0.2** for long-length datasets IMDB and Yelp respectively. For other short-sentence datasets, $\tau = 1$. We set the similarity threshold $\epsilon = 0.7$ to balance attack success rate with semantic preservation, grammaticality, and perturbation rate. For comparison, TextFooler [17] uses a lower threshold (e.g., 0.5) to maximize attack success at the expense of semantic consistency. We found $\epsilon = 0.7$ can provide a more faithful trade-off across various datasets and tasks.

5) **Evaluation Metrics:** (i) **After-attack Accuracy:** The classification accuracy of the target model on generated adversarial examples. A larger drop from the clean-data accuracy indicates a stronger attack. (ii) **Perturbation Rate:** The ratio of substituted words to the total sentence length, reflecting

perturbation magnitude under the L_0 norm. Lower rates imply more imperceptible adversarial examples. (iii) **Grammatical Correctness:** We evaluate grammatical correctness by measuring the increase in grammatical errors between the clean and adversarial texts. *LanguageTool*,⁴ an open-source grammar checker, is used to count grammatical errors for each sentence pair. The grammatical error increase rate is then averaged over the test set, where smaller values denote better fluency preservation. (iv) **Human Evaluation:** Human evaluation was also conducted to ensure that the generated adversarial examples are of a high standard under the

⁴<https://languagetool.org/>

TABLE IV

COMPARISON BETWEEN TEXTFOOLER, ABP, AND OURS ON FINE-TUNED LLaMA-3.2 MODELS. ABBREVIATIONS: ORIG. (ORIGINAL ACCURACY), ACC. ↓ (AFTER-ATTACK ACCURACY), PERT. ↓ (PERTURBATION RATE), GRAM. ↓ (GRAMMATICAL ERROR RATE), USE↑ (SEMANTIC SIMILARITY).
↓ DENOTES LOWER VALUES ARE BETTER; ↑ DENOTES HIGHER VALUES ARE BETTER. BOLD INDICATES BEST PERFORMANCE

Dataset	Attack	LLaMA-3.2-1B					LLaMA-3.2-3B				
		Orig.%	Acc.↓	Pert.↓	Gram.↓	USE↑	Orig.%	Acc.↓	Pert.↓	Gram.↓	USE↑
MR	TextFooler	14.30	12.19	25.49	25.49	0.67	18.10	13.06	65.60	65.60	0.46
	ABP	85.60	31.31	39.43	13.23	0.20	90.00	57.00	20.21	60.71	0.48
	Ours	0.00	9.92	4.77	4.77	0.80	3.70	9.46	4.90	4.90	0.66
IMDB	TextFooler	10.80	9.40	18.95	18.95	0.64	8.83	10.19	16.80	16.80	0.52
	ABP	95.10	50.58	24.64	17.50	0.45	91.70	43.63	46.97	43.18	0.16
	Ours	0.00	3.46	2.40	2.40	0.74	0.69	4.13	2.60	2.60	0.85
Yelp	TextFooler	9.10	9.08	15.33	15.33	0.59	31.76	9.37	25.70	25.70	0.51
	ABP	93.70	56.78	24.19	15.95	0.40	95.30	70.29	23.90	37.22	0.41
	Ours	0.00	4.54	1.46	1.46	0.69	10.13	5.10	6.20	6.20	0.79
AG	TextFooler	17.50	8.79	3.28	3.28	0.53	20.50	22.79	64.60	64.60	0.24
	ABP	87.90	52.90	20.93	21.51	0.55	91.00	70.29	38.56	10.14	0.20
	Ours	4.80	5.36	0.58	0.58	0.71	16.57	10.82	8.70	8.70	0.61
BoolQ	TextFooler	7.90	5.23	7.45	7.45	0.71	28.20	5.33	5.70	5.70	0.71
	ABP	85.10	60.90	19.09	24.82	0.60	78.40	59.86	18.74	39.28	0.47
	Ours	0.40	1.40	5.13	5.13	0.88	1.39	2.36	1.90	1.90	0.83

human perception, including successful prediction accuracy, grammatical correctness assessment and semantic similarity. The human evaluation comprises three criteria: classification accuracy, grammatical correctness, and semantic similarity. More details can be found in Subsection IV-C. (v) **Semantic Similarity:** To quantify meaning preservation, we measure the cosine similarity between sentence embeddings of clean and adversarial examples [41]. USE embeddings are adopted for this purpose, and higher similarity scores indicate better semantic consistency between the two texts.

B. Analysis of Attack Performance Compared with State-of-the-Art and Software Evaluation

In Table II and Table III, our attack yields better overall performance in **after-attack accuracy** and **average perturbation rate** for text classification and entailment. Unlike methods that filter high-perturbation examples, we report all perturbed sentences to avoid bias. After-attack accuracy and perturbation rate drop below 5% and 10%, respectively. For example, on AG attacking BERT our method modifies only 10.17% of words and reduces accuracy from 94.6% to 4.3%, whereas PWWS perturbs 17.2% of words and leaves a much higher after-attack accuracy (37.8%). PWWS's out-of-vocabulary replacements can harm sentence naturalness, while our replace-back stage reduces redundant perturbations. Efficiency results in Subsection IV-D further show competitive runtime. Notably, three models on IMDB are fully misled (0% accuracy) with very low perturbation rates (2.28%, 1.55%, 1.56%), demonstrating strong effectiveness with minimal, visually imperceptible changes. Therefore, our attack is proved to perform fewer modifications with lower after-attack accuracy for visual imperceptibility.

Performance on Fine-tuned LLMs: To further evaluate generalizability, we test our method on fine-tuned LLaMA-3.2-1B and 3.2-3B models (Table IV). Across both sizes and all datasets, our approach consistently surpasses TextFooler and the recent ABP baseline. On LLaMA-3.2-1B, it reduces after-attack accuracy to 0% on MR, IMDB, and Yelp, with the lowest perturbation rates (e.g., 3.46% on IMDB vs. 9.40%

TABLE V
COMPARISON WITH TEXTFOOLER OF USE SCORE (SEMANTIC SIMILARITY). HIGHER SCORES INDICATE BETTER SEMANTIC PRESERVATION

Dataset	Attack	BERT	WordLSTM	WordCNN
MR	TextFooler	0.65	0.67	0.68
	Ours	0.83	0.77	0.71
IMDB	TextFooler	0.86	0.83	0.89
	Ours	0.88	0.87	0.89
Yelp	TextFooler	0.74	0.79	0.82
	Ours	0.80	0.88	0.83
AG	TextFooler	0.57	0.63	0.76
	Ours	0.73	0.73	0.76

for TextFooler and 24.64% for ABP), while preserving fluency and meaning better (USE 0.74 vs. 0.45 for ABP). For the larger LLaMA-3.2-3B, similar trends hold: our method achieves 3.70% accuracy on MR with only 9.46% perturbation, compared to 57.00% and 20.21% for ABP. Although ABP efficiently generates adversarial examples, it is time-consuming due to substitution preference construction. In contrast, our approach attains higher attack success, fewer perturbations, and better linguistic quality without preprocessing overhead.

The **software evaluation** of the **increase in grammatical errors** between original and adversarial samples indicates that our method maintains a low error increase while preserving semantic similarity. Although not always the lowest, slight grammatical errors are natural in real scenarios and often imperceptible to humans, demonstrating satisfactory performance.

Beyond grammaticality, we further assess semantic preservation using the USE score. As shown in Table V, our method consistently achieves higher or comparable scores across models and datasets. For instance, on MR, it improves the USE score from 0.65 to 0.83 under BERT, 0.67 to 0.77 under WordLSTM, and 0.68 to 0.71 under WordCNN. Similar trends appear on Yelp, where our method attains 0.80, 0.88, and 0.83, surpassing TextFooler's 0.74, 0.79, and 0.82, respectively.

TABLE VI

HUMAN-EVALUATION RESULTS OF THE ORIGINAL (ORIG.) AND ADVERSARIAL (ADV.) SENTENCES ON MODEL PREDICTION ACCURACY (ACC.), GRAMMATICAL CORRECTNESS (GRA.) AND SEMANTIC SIMILARITY (SEM.) OF BERT MODEL ON MR AND SNLI DATASETS

Dataset		Acc.	Gra.	Sem.
MR	Orig.	96.5%	4.92	4.18
	Adv.	83.0%	4.77	
SNLI	Orig.	91.0%	4.94	4.01
	Adv.	64.5%	4.52	

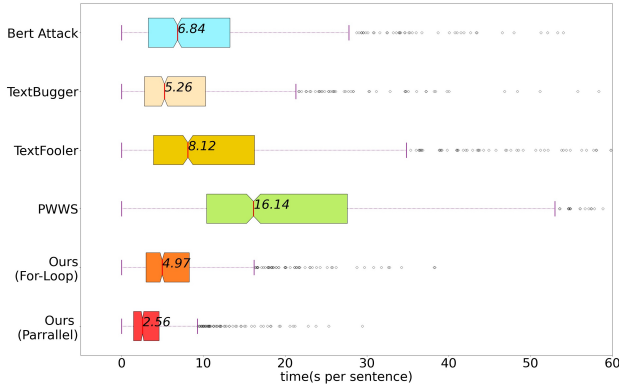


Fig. 3. Box plot of time comparison with baselines.

C. Human Evaluation

We conducted a human evaluation to determine the quality of the generation in terms of *classification accuracy*, *grammatical correctness*, and *semantic similarity* compared with original sentences. The BERT model was targeted on the MR and SNLI datasets using 200 randomly selected test sentences to generate adversarial examples. Using the same operations as [55], we mixed original and adversarial texts and allocate human judges to evaluate their grammatical accuracy and semantic similarity on a Likert scale of 1-5, following the practice as [17]. Before annotation, candidates underwent a test. To determine whether our results are natural, we ask five humans to assign subcategories to each of examples while true labels are hidden and compute the average rates and scores.

As seen in Table VI, the successful classification accuracy for adversarial samples is close to the original accuracy for MR dataset. For SNLI dataset, the gap of accuracy is mainly due to the short length of the hypothesis sentences that restrict the synonym substitution space. The grammatical correctness and semantic similarity for both tasks are relatively high, owing to synonym substitution and POS filter.

D. Ablation Study on Computational Efficiency

Due to the computational complexity linear to the sentence length, the runtime of our attack is competitive in contrast to prior work on BERT model for IMDB dataset. In Fig. 3, our parallel-based attack shows 2-6 times faster than other baselines for per-sentence runtime, while our for-loop strategy is also competitive in terms of efficiency. Furthermore, our parallel-efficient attack achieves 170-1100 times dramatically higher speed than population-based attacks in Table VII.

TABLE VII

COMPARISON WITH POPULATION-BASED ATTACKS

Dataset	Method	Runtime(s/sentence)
IMDB	GA	2835.32
	Hard-label Attack	458.90
	PSO	1203.37
	Ours (Parallel)	2.56

TABLE VIII

THE ORIGINAL ACCURACY ("ORIG.%"), AFTER-ATTACK ACCURACY ("ACC.%"), AVERAGE PERCENTAGE OF PERTURBED WORDS ("PERT.%") AND THE NUMBER OF QUERIES(QUE.) OF ORIGINAL TRAINING ("ORIG.") AND ADVERSARIAL TRAINING ("ADV.") OF BERT ON MR DATASET

Dataset	Train	Bert			
		Orig.%	Acc.%	Pert.%	Que.
MR	Orig.	92.2	10.1	12.8	155
	Adv.	85.6	15.5	13.938	165.1

We perform the computation on a PC with i7-8559U CPU, 16GB of RAM and NVIDIA GeForce RTX 2080 Ti GPU. Additionally, time complexity analysis is further provided in the next subsection.

E. Complexity Analysis

As shown in Figure 3 and Table VII in Subsection IV-D, our attack demonstrates superior runtime efficiency compared to baseline methods. This subsection analyzes the time complexity of SCALA under both sequential and parallel strategies.

After Step-1 filtering, let q denote the number of eligible non-meaningless words, z the average number of synonym candidates per word, and $\tau \in [0, 1]$ the adaptive parameter controlling synonym quantity. Step-2 performs one-word resorting in $O(\tau q z)$ time. Based on the sorted indices, accumulated perturbations are computed in $O(q)$ time. Step-4 (replace-back reordering) and redundancy elimination each require $O(q)$ time. Thus, the overall sequential complexity is $O((\tau z + 3)q)$.

For the parallel version, assuming a batch size of η , the total complexity becomes $O((\tau z + 3)q/\eta)$. This design allows flexible adjustment of τ for longer sentences to maintain high efficiency. Even with a small τ , SCALA effectively generates adversarial examples with low after-attack accuracy and perturbation rates. For example, $\tau = 0.06$ on IMDB (attacking BERT) yields 0% after-attack accuracy with only 2.28% perturbation, as shown in Table II of Subsection IV-B.

F. Adversarial Transferability

To evaluate **transferability**, we select adversarial examples misclassified by a target model and re-test them on other models. If adversarial samples crafted for one model can also fool others, the attack exhibits good transferability. This property indicates whether an attack can succeed against a DNN model without direct access to it. We conduct experiments on the IMDB and SNLI datasets for text classification and textual entailment. The resulting after-attack accuracies are reported in Tables IX and X. Notably, higher transferability corresponds to lower after-attack accuracy. Compared with the TextFooler baseline, our method demonstrates substantially better transferability across multiple datasets.

TABLE IX
TRANSFERABILITY COMPARISON AFTER-ATTACK ACCURACY
WITH TEXTFOOLER ON IMDB DATASET

Gen.	Test			
	Attack	wordCNN	wordLSTM	BERT
wordCNN	TF	-	84.9%	90.20%
	Ours	-	18.02%	4.01%
wordLSTM	TF	74.9%	-	87.9%
	Ours	34.80%	-	10.09%
BERT	TF	84.1%	85.1%	-
	Ours	12.2%	11.4%	-

TABLE X
TRANSFERABILITY COMPARISON WITH TEXTFOOLER ON SNLI DATASET

Gen.	Test			
	Attack	BERT	InferSent	ESIM
BERT	TF	-	58.2%	54.6%
	Ours	-	58.5%	61.1%
InferSent	TF	67.7%	-	62.7%
	Ours	17.6%	-	20.5%
ESIM	TF	59.3%	49.4%	-
	Ours	16.9%	19.4%	-

TABLE XI
CRITERIA FOR EVALUATING GRAMMATICAL CORRECTNESS AND SEMANTIC SIMILARITY. SCORES: 1 (POOR) TO 5 (EXCELLENT)

Score	Grammar Correctness	Semantic Similarity
1	Ungrammatical; sentence structure is broken or unreadable	No similarity in meaning; adversarial sentence is unrelated
2	Major grammatical issues; hard to understand	Slightly related, but meaning significantly altered or lost
3	Understandable but contains moderate grammar errors	Partially similar; some key meaning retained, but altered
4	Mostly correct grammar; minor fluency or tense issues	Largely preserves original meaning with small semantic shifts
5	Fully grammatical and fluent	Nearly identical meaning to original; paraphrase-level similarity

G. Adversarial Training

We further adopt adversarial training as a defence strategy to evaluate the effectiveness of our attack. We obtained all the adversarial examples that deceived BERT from the MR training sets and mixed them with the original training set. During an initial trial of adversarial training, we fed the models both clean sentences and adversarial examples. The labels of generated adversarial samples are processed back to their initial counterparts for observing whether this operation can contribute to strengthening model robustness. Then we train BERT with the augmented data. As shown in Table VIII, we recalculate after-attack accuracy, perturbation rate and average queries, which show increases of 5.4%, 1.2% and 10 respectively. It comes as no surprise that adding adversarial samples to the training data improves the target model's robustness to attacks, since it requires more perturbations and queries.

TABLE XII
REPRESENTATIVE FAILURE CASES FOR SCALA

Original Sentence	Label	Failure Type	Reason for SCALA Ineffectiveness
Great character interaction	1	Short phrase with strong meaning	Substituting "great" or "interaction" alters the core sentiment; few paraphrasing options without drift.
A ragbag of cliches	0	Idiomatic / uncommon phrase	"Ragbag" is an idiom; substitutions fail to preserve the idiomatic tone and negativity.
Good, solid storytelling	1	Low redundancy	"Good" and "solid" are key sentiment tokens; replacing either weakens the intended meaning.
Too contrived to be as naturally charming as it needs to be	0	Dense sentence structure	Nuanced and compound expression; substitutions tend to break semantic coherence.
Reeks of rot and hack work from start to finish	0	Specific sentiment words	Strongly negative words like "reeks" and "rot" have no neutral or soft equivalents that preserve intent.

H. Analysis of Human Evaluation Process and Failure Cases

We highlight several insights regarding synonym quality, semantic preservation, and structural limitations that affect attack success. We use a 1–5 Likert scale in Table XI to evaluate semantic similarity and grammatical correctness between original and adversarial examples. As seen in Table XIII, generated examples with high and low scores are listed with reasons. SCALA maintains higher fluency and relevance than TextFooler. We categorize common failure modes where SCALA adversarial generation is less effective in Table XII. These cases reveal the trade-off in our attack: preserving naturalness and semantics is feasible when redundancy or synonym coverage is sufficient. However, adversarial success declines for idiomatic or sentiment-critical constructs.

I. Examples of Adversarial Samples

This section present successfully generated examples of adversarial samples on text classification. Table XIV encompasses datasets from Yelp, AG and MR for Bert, CNN, and LSTM models. Only a small quantity of words are perturbed with synonyms, leading to changes in prediction labels for NLP models. Notably, the generated sentences exhibit visual and semantic similarity, aligning with human perception.

V. LIMITATIONS AND DESIGN TRADE-OFFS

While SCALA demonstrates strong performance across diverse datasets and models, several limitations remain. First, our method has not been systematically evaluated against adversarial defense techniques such as adversarial training, input preprocessing (e.g., spell-checking or paraphrasing), or defense-aware robust embeddings. Incorporating these defenses in future experiments would enable a more comprehensive assessment of SCALA's robustness in defense-aware settings. Second, SCALA depends on the quality and contextual fit of synonym substitutions. When suitable replacements are limited or contextually inappropriate, the attack's effectiveness may decrease due to reduced naturalness or semantic drift. Third, the current implementation is limited to

TABLE XIII

ADVERSARIAL EXAMPLES OF TEXTFOOLER AND SCALA. GRAMMAR AND SEMANTIC SIMILARITY ARE RATED FROM 1 TO 5, THE HIGHER THE BETTER

Original	Variant	Grammar	Semantic	Reasoning
Label:1 Much of what we see is horrible but it's also undeniably exceedingly clever	SCALA: Label:0 Much of what we see is horrible but it's also undeniably overly clever TextFooler: Label:0 Comically of what we clotted is vano but it's also broward tingly clever	5 1	5 1	Fluent; subtle tone shift from 'exceedingly' to 'overly' slightly softens sentiment. Nonsensical and ungrammatical; 'clotted', 'vano', 'broward' are invalid.
Label:1 This is one of those war movies that focuses on human interaction rather than battle and action sequences and it's all the stronger because of it.	SCALA: Label:0 This is one of those war movies that focuses on human interaction rather than battle and action sequences and it's all the stiffer because of it. TextFooler: Label:0 This is one of those war movies that focuses on schlegel interaction rather than battle and action sequences and it's all the wracked because of it.	5 3	4 3	Grammatically correct; 'stiffer' is unusual but acceptable; slight semantic shift from 'stronger'. 'Schlegel' and 'wracked' are contextually meaningless; sentence is partially broken.
Label:0 this one aims for the toilet and scores a direct hit	SCALA: Label:1 this one aims for the loo and scores a direct hit TextFooler: Label:1 this one aims for the fawn and scores a direct hit	5 3	5 2	Fully grammatical; 'loo' is British informal for 'toilet'. 'Fawn' is off-topic, breaks metaphor.
Label:1 by the standards of knucklehead swill, the hot chick is pretty damned funny	SCALA: Label:0 by the standards of knucklehead swill , the hot chick is pretty damned humor humor TextFooler: Label:0 by the deferment of knucklehead swill, the hot chick is pretty damned funny	4 3	3 2	'Humor' as noun disrupts expected adjective. Misuse of 'deferment'; unclear meaning.

TABLE XIV

ADVERSARIAL EXAMPLES FOR TEXT CLASSIFICATION TASKS. BLUE: THE PREDICTION LABELS; RED: THE PERTURBED WORDS

Yelp for Bert (Labels: Positive, Negative)	
Orig. (pos.)	Great place for lunch service is good.
Adv. (neg.)	Great sites for lunch service is good.
AG's News for CNN (Labels: World, Sports, Business, Sci/Tech)	
Orig. (Business)	Barrett good business models vary widely Bangalore, India peer to peer <i>p2p</i> sharing would never have gathered momentum if the music industry had adopted models for distribution over the internet, said intel , corp chief executive officer Craig Barrett, addressing it executives in India Friday.
Adv. (Sports)	Barrett good business models vary widely Bangalore, India peer to peer <i>p2p</i> sharing would never have gathered momentum if the music industry had adopted models for distribution over the internet, avowed matushita corp chief executive officer Craig Barrett, addressing it executives in India Friday.
MR for LSTM (Labels: Positive, Negative)	
Orig. (pos.)	this flick is about as cool and crowd pleasing as a documentary can get.
Adv. (neg.)	this flick is about as cool and crowd loveable as a documentary can get.

English, constrained by the availability of high-quality linguistic resources. Extending SCALA to multilingual scenarios is a promising direction for future work. Finally, SCALA does not employ exhaustive combination-based synonym substitution strategies. Although such methods can identify perturbations with minimal changes, they are computationally expensive and impractical for real-time attack settings. Our experiments show that exhaustive combination approaches introduce significant overhead, thus not suitable for long-text datasets such as IMDB. In contrast, SCALA prioritizes efficiency, achieving **100–700× faster** runtime than population-based methods like GA [18], PSO [27], and hard-label attacks [42], while maintaining comparable or superior attack performance (see Table VII and Subsections IV-D and IV-E).

VI. CONCLUSION

In this work, we proposed SCALA, a score-based black-box adversarial attack that achieves an effective balance between imperceptibility, efficiency, and practicality. By leveraging a novel descending–ascending synonym ranking mechanism with parallelized computation, SCALA generates visually and semantically natural adversarial texts with minimal perturbations. Extensive experiments on both conventional NLP

models and fine-tuned LLMs demonstrate the generality and scalability of our method. SCALA consistently achieves the lowest perturbation and grammatical error rates while maintaining high semantic similarity, outperforming representative baselines. The results highlight that fine-tuned LLMs, despite their scale and contextual capacity, remain vulnerable to word-level perturbations, emphasizing the need for robust defense strategies. Future work will explore extending SCALA to multilingual and multimodal domains and investigating defense-aware training to enhance robustness in real-world applications.

REFERENCES

- [1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [2] L. Ma et al., "ConCare: Personalized clinical feature embedding via capturing the healthcare context," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, 2020, pp. 833–840.
- [3] T. Le, S. Wang, and D. Lee, "MALCOM: Generating malicious comments to attack neural fake news detection models," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2020, pp. 282–291.
- [4] C. Szegedy et al., "Intriguing properties of neural networks," 2013, *arXiv:1312.6199*.
- [5] F. Wang, C. Zhang, P. Xu, and W. Ruan, "Deep learning and its adversarial robustness: A brief introduction," in *Handbook on Computer Learning and Intelligence: Deep Learning, Intelligent Control and Evolutionary Computation*, vol. 2, 2022, pp. 547–584.
- [6] X. Huang et al., "A survey of safety and trustworthiness of deep neural networks: Verification, testing, adversarial attack and defence, and interpretability," *Comput. Sci. Rev.*, vol. 37, Aug. 2020, Art. no. 100270.
- [7] W. Wang, R. Wang, L. Wang, Z. Wang, and A. Ye, "Towards a robust deep neural network in texts: A survey," 2019, *arXiv:1902.07285*.
- [8] E. Shayegani, M. A. Al Mamun, Y. Fu, P. Zaree, Y. Dong, and N. Abu-Ghazaleh, "Survey of vulnerabilities in large language models revealed by adversarial attacks," 2023, *arXiv:2310.10844*.
- [9] X. Dong, A. T. Luu, R. Ji, and H. Liu, "Towards robustness against natural language word substitutions," 2021, *arXiv:2107.13541*.
- [10] B. C. Das, M. H. Amini, and Y. Wu, "Security and privacy challenges of large language models: A survey," *ACM Comput. Surveys*, vol. 57, no. 6, pp. 1–39, Jun. 2025.
- [11] R. Moraffah and H. Liu, "Query-efficient target-agnostic black-box attack," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2022, pp. 368–377.
- [12] W. Y. Wang, J. Li, and X. He, "Deep reinforcement learning for NLP," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics, Tutorial Abstr.*, 2018, pp. 19–21.
- [13] X. Gao, Z. Zhu, X. Chu, Y. Wang, W. Ruan, and J. Zhao, "Enhancing robust text classification via category description," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Sep. 2022, pp. 151–160.

- [14] J. Li, S. Ji, T. Du, B. Li, and T. Wang, "TextBugger: Generating adversarial text against real-world applications," in *Proc. 26th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2019, pp. 1–15.
- [15] D. Pruthi, B. Dhingra, and Z. C. Lipton, "Combating adversarial misspellings with robust word recognition," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 5582–5591.
- [16] S. Ren, Y. Deng, K. He, and W. Che, "Generating natural language adversarial examples through probability weighted word saliency," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 1085–1097.
- [17] D. Jin, Z. Jin, J. T. Zhou, and P. Szolovits, "Is BERT really robust? A strong baseline for natural language attack on text classification and entailment," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, Apr. 2020, pp. 8018–8025.
- [18] M. Alzantot, Y. Sharma, A. Elgohary, B.-J. Ho, M. Srivastava, and K.-W. Chang, "Generating natural language adversarial examples," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2018, pp. 2890–2896.
- [19] L. Li, R. Ma, Q. Guo, X. Xue, and X. Qiu, "BERT-ATTACK: Adversarial attack against BERT using BERT," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6193–6202.
- [20] X. Huang et al., "A survey of safety and trustworthiness of large language models through the lens of verification and validation," *Artif. Intell. Rev.*, vol. 57, no. 7, p. 175, Jun. 2024.
- [21] Y. Dong et al., "Safeguarding large language models: A survey," *Artif. Intell. Rev.*, vol. 58, no. 12, p. 382, Oct. 2025.
- [22] J. Ebrahimi, A. Rao, D. Lowd, and D. Dou, "HotFlip: White-box adversarial examples for text classification," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics*, vol. 2, 2018, pp. 31–36.
- [23] B. Liang, H. Li, M. Su, P. Bian, X. Li, and W. Shi, "Deep text classification can be fooled," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, Jul. 2018, pp. 4208–4215.
- [24] S. Garg and G. Ramakrishnan, "BAE: BERT-based adversarial examples for text classification," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6174–6181.
- [25] Z. Meng and R. Wattenhofer, "A geometry-inspired attack for generating natural language adversarial examples," in *Proc. 28th Int. Conf. Comput. Linguistics*, 2020, pp. 6679–6689.
- [26] P. Vijayaraghavan, D. Roy, P. Vijayaraghavan, and D. Roy, "Generating black-box adversarial examples for text classifiers using a deep reinforced model," in *Proc. Mach. Learn. Knowl. Discovery Databases, Eur. Conf.*, Würzburg, Germany, Sep. 2020, pp. 711–726.
- [27] Y. Zang et al., "Word-level textual adversarial attacking as combinatorial optimization," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 6066–6080.
- [28] Z. Yu, Z. Chen, and K. He, "Query-efficient textual adversarial example generation for black-box attacks," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, 2024, pp. 556–569.
- [29] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *Proc. NeurIPS ML Saf. Workshop*, 2022, pp. 1–21.
- [30] Y. Liu et al., "Prompt injection attack against LLM-integrated applications," 2023, *arXiv:2306.05499*.
- [31] D. Kang, X. Li, I. Stoica, C. Guestrin, M. Zaharia, and T. Hashimoto, "Exploiting programmatic behavior of LLMs: Dual-use through standard security attacks," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2024, pp. 132–143.
- [32] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, "'Do anything now': Characterizing and evaluating in-the-wild jailbreak prompts on large language models," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 1671–1685.
- [33] G. Deng et al., "MASTERKEY: Automated jailbreaking of large language model chatbots," in *Proc. NDSS*, 2024, pp. 1–16.
- [34] J. Yu, X. Lin, Z. Yu, and X. Xing, "GPTFUZZER: Red teaming large language models with auto-generated jailbreak prompts," 2023, *arXiv:2309.10253*.
- [35] X. Liu, N. Xu, M. Chen, and C. Xiao, "AutoDAN: Generating stealthy jailbreak prompts on aligned large language models," in *Proc. 12th Int. Conf. Learn. Represent.*, 2023, pp. 1–12.
- [36] H. Zhao, C. Ma, X. Dong, A. T. Luu, Z. Deng, and H. Zhang, "Certified robustness against natural language attacks by causal intervention," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 26958–26970.
- [37] S. Sun and W. Ruan, "TextVerifier: Robustness verification for textual classifiers with certifiable guarantees," in *Proc. Findings Assoc. Comput. Linguistics: ACL*, 2023, pp. 4362–4380.
- [38] S. Sun, P. Sen, and W. Ruan, "CROWD: Certified robustness via weight distribution for smoothed classifiers against backdoor attack," in *Proc. Findings Assoc. Comput. Linguistics: EMNLP*, 2024, pp. 17056–17070.
- [39] W. Ruan, M. Wu, Y. Sun, X. Huang, D. Kroening, and M. Kwiatkowska, "Global robustness evaluation of deep neural networks with provable guarantees for the Hamming distance," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 5944–5952.
- [40] D. Cer et al., "Universal sentence encoder," 2018, *arXiv:1803.11175*.
- [41] N. Mrkšić et al., "Counter-fitting word vectors to linguistic constraints," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, 2016, pp. 142–148.
- [42] R. Maheshwary, S. Maheshwary, and V. Pudi, "Generating natural language attacks in a hard label black box setting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 15, pp. 13525–13533.
- [43] B. Pang and L. Lee, "Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales," in *Proc. 43rd Annu. Meet. Assoc. Comput. Linguist.*, 2005, pp. 115–124.
- [44] A. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proc. 49th Annu. Meeting Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2011, pp. 142–150.
- [45] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 649–657.
- [46] C. Clark, K. Lee, M. Chang, T. Kwiatkowski, M. J. Collins, and K. Toutanova, "BoolQ: Exploring the surprising difficulty of natural yes/no questions," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2019, pp. 2924–2936.
- [47] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, "A large annotated corpus for learning natural language inference," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2015, pp. 632–642.
- [48] A. Williams, N. Nangia, and S. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, 2018, pp. 1112–1122.
- [49] A. Grattafiori et al., "The llama 3 herd of models," 2024, *arXiv:2407.21783*.
- [50] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [51] Y. Kim, "Convolutional neural networks for sentence classification," 2014, *arXiv:1408.5882*.
- [52] Q. Chen, X. Zhu, Z. Ling, S. Wei, H. Jiang, and D. Inkpen, "Enhanced LSTM for natural language inference," 2016, *arXiv:1609.06038*.
- [53] A. Conneau, D. Kiela, H. Schwenk, L. Barrault, and A. Bordes, "Supervised learning of universal sentence representations from natural language inference data," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2017, pp. 670–680.
- [54] J. X. Morris, E. Lifland, J. Y. Yoo, J. Grigsby, D. Jin, and Y. Qi, "TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP," in *Proc. Conf. Empirical Methods Natural Lang. Process., Syst. Demonstrations*, 2020, pp. 119–126.
- [55] J. Gagnon-Marchand, H. Sadeghi, M. A. Haidar, and M. Rezagholizadeh, "SALSA-TEXT: Self attentive latent space based adversarial text generation," in *Proc. Can. Conf. Artif. Intell. Cham, Switzerland: Springer*, 2019, pp. 119–131.